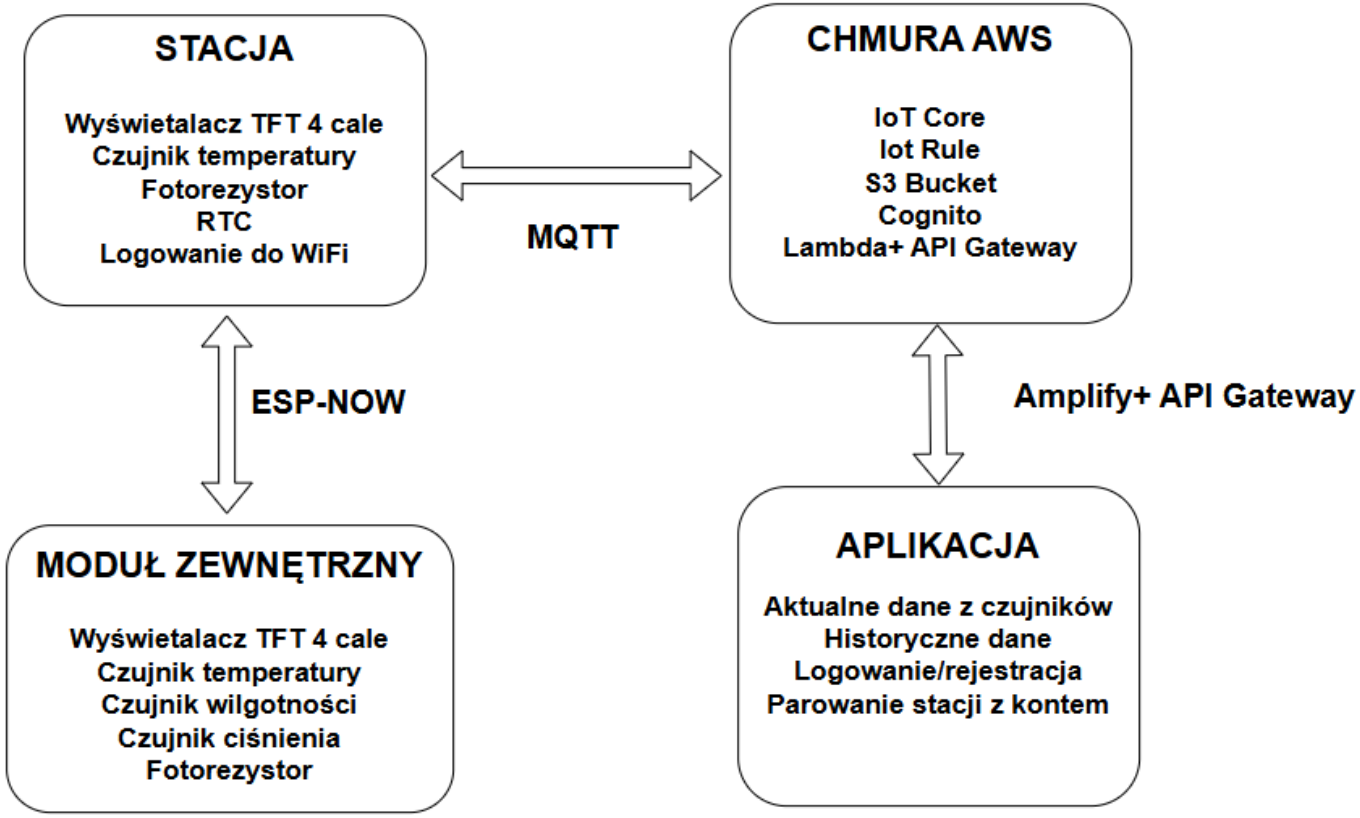
STACJA + MODUŁ ZEWNĘTRZNY, CHMURA I APLIKACJA WEBOWA

Autor: Łukasz Wolf

OGÓLNY SCHEMAT SYSTEMU



PRZYSTOSOWANIE PRODUKTU DO PRODUKCJI

AWS tworzy dla każdego konta użytkownika i podłączonej do niego stacji nowy folder w bazie danych S3 Bucket.

Użytkownik po zalogowaniu widzi tylko dane z stacji przypisanej do swojego konta.

Urządzenia na etapie produkcji są dodawane do Iot Core i łączone z chmurą na stałe przy pomocy certyfikatów

MODUŁ ZEWNĘTRZNY

- Mikrokontroler ESP32 lub inny mniejszy zamiennik
- Czujniki temperatury, ciśnienia oraz wilgotności
- Deep Sleep (wysyłanie danych do stacji co 10 minut) przy pomocy ESP-Now
- Zasilanie z baterii

STACJA - ZASILANIE

- Zasilanie z sieci przez zewnętrzny zasilacz 230/5V
- Port USB
- Sekcja zasilania na płytce PCB (elementy filtrujące i stabilizujące)

STACJA - CZUJNIKI

Czujnik temperatury – pomiar temperatury wewnątrz budynku

Fotorezystor – opcjonalne automatyczne dostosowywanie jasności ekranu

RTC – moduł czasu rzeczywistego , dane o czasie, dacie itp.

Dotykowy wyświetlacz TFT - główny interfejs użytkownika zapewniający interakcję ze stacją

WYŚWIETLACZ

- Rozmiar: 4 cale
- Podłączony do ESP32 przez magistralę SPI
- Obsługa dotyku
- Interaktywna konfiguracja ustawień systemu
- Wyświetlanie danych z czujników , aktualnego czasu
- Regulowana jasność ekranu

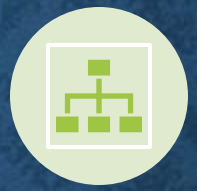
STACJA - PIERWSZE POŁĄCZENIE Z WIFI

ESP32 znajdujące się w stacji zapamiętuje w swojej pamięci dane do sieci WiFi i próbuje się do niej zalogować. Jeśli to się nie uda to uruchamia serwer HTPP ze stroną internetową umożliwiającą zalogowanie do sieci.

AMAZON WEB SERVICES

PRZECHOWYWANIE DANYCH (S3 BUCKET)

SERVERLESS BACKEND DO APLIKACJI (API GATEWAY ORAZ LAMBDA)



ZARZĄDZANIE KONTAMI UŻYTKOWNIKÓW (COGNITO)

IOT CORE – BAZA DANYCH URZĄDZEŃ

IOT RULES - REGUŁY DO ODCZYTU I ZAPISU DANYCH PRZEZ UŻYTKOWNIKÓW

CERTYFIKATY ORAZ ZABEZPIECZENIA DANYCH I KOMUNIKACJI

APLIKACJA

- Aplikacja internetowa React
- Hosting Vercel (dostęp z każdego miejsca na ziemi)
- Logowanie, rejestracja użytkownika
- Aktualne pomiary
- Dane historyczne
- Parowanie konta z stacją
- Wykorzystuje usługi AWS

KOSZTY

Usługi AWS: przy małym ruchu na stronie i niewielkiej ilości danych w S3 koszty zamykają się w kilku złotych na rok

Hosting: Vercel jest darmowy

Szacowany maksymalny koszt elementów: około 200zł

ŚRODOWISKO PROGRAMISTYCZNE

- VS Code (dla części związanej z aplikacją)
- PlatformIO (dla części sprzętowej) - Rozszerzenie do VS Code przeznaczone do tworzenia aplikacji na systemy wbudowane

Visual Studio Code

File Edit Selection View Go Run Terminal Help

meteo-app

EXPLORER

- METEO-APP
  - meteo-dashboard
    - dist
    - node_modules
    - public
    - src
      - assets
      - App.css
      - App.jsx
      - aws-exports.js
      - index.css
      - main.jsx
    - .gitignore
    - eslint.config.js
    - index.html
    - package-lock.json
    - package.json
    - README.md
    - vite.config.js

OUTLINE

- #root
- .logo
- .logo:hover
- .logo.react:hover
- @keyframes logo-spin

TIMELINE

App.css

meteo-dashboard > src > App.css > .logo

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
```

OUTPUT DEBUG CONSOLE TERMINAL PROBLEMS PORTS

esbuild - meteo-dashboard

```
PS C:\Users\MSI\Desktop\meteo-app\meteo-dashboard> npm run dev

  VITE v7.1.9  ready in 1293 ms

  ➜  Local:   http://localhost:5173/meteo-app/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

CHAT

Build with agent mode

AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

App.css

Add context (#), extensions (@), commands

Agent Auto

main ⊗ 0 ⚠ 0 Live Share

LukaszWolf (1 week ago) Ln 9, Col 15 Spaces: 2 UTF-8 LF CSS

PlatformIO

PlatformIO Core 6.0

EXPLORER

METEO-STATION
- .pio
- .vscode
  - c_cpp_properties.json
  - extensions.json
  - launch.json
- data
  - icons
  - setup
  - app.js
  - index.html
  - manifest.json
  - style.css
  - sw.js
- include
- lib
- src
  - main.cpp
- test
- .gitignore
- AmazonRootCA1.pem
- f54678f0821b8ec25458a23f24c15e357cf24b...
- f54678f0821b8ec25458a23f24c15e357cf24b...
- partitions.csv
- platformio.ini

OUTLINE

TIMELINE

platformio.ini | main.cpp | PIO Home

platformio.ini

```
[env:esp32doit-devkit-v1]
platform = espressif32
board = esp32doit-devkit-v1
framework = arduino
monitor_speed = 115200
upload_port = COM3
monitor_port = COM3

; LittleFS w core Arduino-ESP32 v3
board_build.filesystem = littlefs

lib_deps =
    https://github.com/esphome/ESPAsyncWebServer.git#v3.4.0
    https://github.com/esphome/AsyncTCP.git#v2.1.4
    https://github.com/khoih-prog/ESPAsyncDNSServer.git
    bblanchon/ArduinoJson@^7
    knolleary/PubSubClient @ ^2.8
```

OUTPUT | DEBUG CONSOLE | TERMINAL | PROBLEMS | PORTS

powershell

```
PS C:\Users\MSI\Documents\PlatformIO\Projects\meteo-station> pio run -t upload
```

CHAT

Build with agent mode

AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

platformio.ini

Add context (#), extensions (@), commands

Agent | Auto

Dziękuję za uwagę!